

MySQL Commands

Complete Reference Guide

Category	Commands	Use case
1. Database	CREATE, USE, SHOW, DROP	Manage databases
2. Table	CREATE, ALTER, DROP, DESCRIBE	Define schema
3. CRUD	INSERT, SELECT, UPDATE, DELETE	Data operations
4. Query	WHERE, GROUP BY, CASE WHEN	Filter & aggregate
5. Joins	INNER, LEFT, UNION	Combine tables
6. Index	CREATE INDEX, EXPLAIN	Performance tuning
7. User	CREATE USER, GRANT, REVOKE	Access control
8. Transaction	BEGIN, COMMIT, ROLLBACK	Data integrity
9. Admin	BACKUP, PROCEDURE, TRIGGER, VIEW	Management

For Java Spring Boot Banking Applications — MySQL 8.0+

1

Database Commands

Database

CREATE DATABASE

Create a new database

```
CREATE DATABASE bank_db;  
CREATE DATABASE bank_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

i Always specify utf8mb4 for full Unicode support including emojis.

USE

Switch to a database

```
USE bank_db;
```

i Run this before any table operations so MySQL knows which database to work in.

SHOW DATABASES

List all databases

```
SHOW DATABASES;  
SHOW DATABASES LIKE 'bank%';
```

DROP DATABASE

Delete a database permanently

```
DROP DATABASE bank_db;  
DROP DATABASE IF EXISTS bank_db;
```

i Warning: deletes everything. Always backup first. IF EXISTS prevents error if DB does not exist.

SELECT DATABASE()

Show current active database

```
SELECT DATABASE();
```

2

Table Commands

Table

CREATE TABLE

Create a new table

```
CREATE TABLE accounts (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  account_no  VARCHAR(20)    NOT NULL UNIQUE,  
  name        VARCHAR(100)   NOT NULL,  
  balance     DECIMAL(15,2)  DEFAULT 0.00,  
  status      ENUM('ACTIVE', 'INACTIVE', 'CLOSED') DEFAULT 'ACTIVE',  
  created_at  DATETIME       DEFAULT CURRENT_TIMESTAMP,  
  updated_at  DATETIME       DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

i Use *DECIMAL* for money -- never *FLOAT* or *DOUBLE*. *AUTO_INCREMENT* gives surrogate primary keys.

ALTER TABLE

Modify existing table structure

```
ALTER TABLE accounts ADD COLUMN email VARCHAR(100);  
ALTER TABLE accounts MODIFY COLUMN name VARCHAR(200) NOT NULL;  
ALTER TABLE accounts DROP COLUMN email;  
ALTER TABLE accounts RENAME COLUMN account_no TO acc_number;  
ALTER TABLE accounts ADD CONSTRAINT fk_branch  
  FOREIGN KEY (branch_id) REFERENCES branches(id);
```

DROP TABLE / TRUNCATE

Delete table or clear all rows

```
DROP TABLE accounts;  
DROP TABLE IF EXISTS accounts;  
TRUNCATE TABLE accounts;
```

i *DROP* removes the table. *TRUNCATE* removes all rows but keeps structure. *TRUNCATE* is faster than *DELETE* for clearing all rows.

DESCRIBE / SHOW

View table structure and details

```
DESCRIBE accounts;  
SHOW COLUMNS FROM accounts;  
SHOW TABLES;  
SHOW CREATE TABLE accounts;  
SHOW TABLE STATUS LIKE 'accounts';
```

i *SHOW CREATE TABLE* shows the exact SQL used to create the table -- useful for replication.

RENAME TABLE

Rename a table

```
RENAME TABLE accounts TO bank_accounts;  
RENAME TABLE old_name TO new_name, another TO another_new;
```

3

CRUD Commands

CRUD

INSERT

Add new rows to a table

```
-- Single row
INSERT INTO accounts (account_no, name, balance)
VALUES ('ACC001', 'John Doe', 5000.00);

-- Batch insert
INSERT INTO accounts (account_no, name, balance) VALUES
('ACC002', 'Jane Smith', 10000.00),
('ACC003', 'Bob Johnson', 2500.00);

-- Insert from another table
INSERT INTO accounts (account_no, name)
SELECT emp_id, full_name FROM employees WHERE dept = 'Finance';
```

i Batch inserts are much faster than individual inserts in loops.

SELECT

Query and retrieve data

```
SELECT * FROM accounts;
SELECT id, name, balance FROM accounts;
SELECT name, balance FROM accounts WHERE status = 'ACTIVE';
SELECT name, balance FROM accounts WHERE balance > 1000 ORDER BY balance DESC;
SELECT name, balance FROM accounts LIMIT 10 OFFSET 20;
SELECT COUNT(*), SUM(balance), AVG(balance) FROM accounts;
```

i Avoid `SELECT *` in production -- always name the columns you need for better performance.

UPDATE

Modify existing rows

```
UPDATE accounts SET balance = 6000.00 WHERE id = 1;

UPDATE accounts
SET balance = balance - 500.00, updated_at = NOW()
WHERE account_no = 'ACC001' AND status = 'ACTIVE';

UPDATE accounts SET status = 'INACTIVE'
WHERE last_transaction_date < DATE_SUB(NOW(), INTERVAL 1 YEAR);
```

i Always include a `WHERE` clause -- without it you update every row in the table.

DELETE

Remove rows from a table

```
DELETE FROM accounts WHERE id = 1;
DELETE FROM accounts WHERE status = 'CLOSED';

DELETE a FROM accounts a
  JOIN customers c ON a.customer_id = c.id
 WHERE c.status = 'DELETED';
```

i Always use *WHERE*. Run a *SELECT* first with the same *WHERE* to preview what will be deleted.

ON DUPLICATE KEY UPDATE

Insert or update (upsert)

```
INSERT INTO accounts (account_no, name, balance)
VALUES ('ACC001', 'John Doe', 5000.00)
ON DUPLICATE KEY UPDATE
  balance = VALUES(balance),
  updated_at = NOW();
```

i Safer than *REPLACE* -- updates rather than deleting and re-inserting.

WHERE Clauses

Filter rows with conditions

```
SELECT * FROM accounts WHERE balance BETWEEN 1000 AND 5000;
SELECT * FROM accounts WHERE name LIKE 'John%';
SELECT * FROM accounts WHERE status IN ('ACTIVE', 'INACTIVE');
SELECT * FROM accounts WHERE email IS NULL;
SELECT * FROM accounts WHERE balance > 1000 AND status = 'ACTIVE';
```

i *LIKE with a leading % (e.g. '%Smith') cannot use an index -- causes a full table scan.*

GROUP BY / HAVING

Aggregate and filter groups

```
SELECT status, COUNT(*) as total, SUM(balance) as total_balance
FROM accounts GROUP BY status;

SELECT branch_id, AVG(balance) as avg_balance
FROM accounts
GROUP BY branch_id
HAVING AVG(balance) > 5000
ORDER BY avg_balance DESC;
```

i *HAVING filters after grouping. WHERE filters before grouping. Use WHERE when possible -- it is faster.*

ORDER BY / LIMIT

Sort and paginate results

```
SELECT * FROM accounts ORDER BY balance DESC;
SELECT * FROM accounts ORDER BY name ASC, balance DESC;
SELECT * FROM accounts ORDER BY balance DESC LIMIT 10;
SELECT * FROM accounts ORDER BY id LIMIT 10 OFFSET 20;
```

i *For pagination in large tables use WHERE id > last_seen_id LIMIT N instead of OFFSET.*

CASE WHEN

Conditional logic in queries

```
SELECT name, balance,
CASE
    WHEN balance >= 10000 THEN 'Platinum'
    WHEN balance >= 5000  THEN 'Gold'
    WHEN balance >= 1000  THEN 'Silver'
    ELSE 'Standard'
END AS account_tier
FROM accounts;
```

Date Functions

Work with dates and times

```
SELECT NOW(); -- current datetime
SELECT CURDATE(); -- current date only
SELECT DATE_FORMAT(created_at, '%Y-%m-%d') FROM accounts;
SELECT DATEDIFF(NOW(), created_at) as days_old FROM accounts;
SELECT * FROM transactions
    WHERE created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY);
```

String Functions

Manipulate string values

```
SELECT UPPER(name), LOWER(email) FROM accounts;
SELECT CONCAT(first_name, ' ', last_name) as full_name FROM customers;
SELECT LENGTH(account_no), TRIM(name) FROM accounts;
SELECT SUBSTRING(account_no, 1, 3) as prefix FROM accounts;
SELECT REPLACE(phone, '-', '') as clean_phone FROM customers;
```


INNER JOIN

Return matching rows from both tables

```
SELECT a.account_no, a.balance, c.name, c.email
FROM accounts a
INNER JOIN customers c ON a.customer_id = c.id
WHERE a.status = 'ACTIVE'
ORDER BY a.balance DESC;
```

i *INNER JOIN only returns rows where the join condition matches in BOTH tables.*

LEFT JOIN

All left rows, matching right rows

```
SELECT c.name, a.account_no, a.balance
FROM customers c
LEFT JOIN accounts a ON c.id = a.customer_id
ORDER BY c.name;
```

```
-- Find customers with NO accounts
SELECT c.name FROM customers c
LEFT JOIN accounts a ON c.id = a.customer_id
WHERE a.id IS NULL;
```

i *The second query finds customers with NO accounts -- a very useful pattern.*

Multiple JOINS

Join three or more tables

```
SELECT t.id, t.amount, t.created_at,
       a.account_no, c.name as customer_name,
       b.name as branch_name
FROM transactions t
INNER JOIN accounts a ON t.account_id = a.id
INNER JOIN customers c ON a.customer_id = c.id
INNER JOIN branches b ON a.branch_id = b.id
WHERE t.created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
ORDER BY t.created_at DESC;
```

i *Alias your tables (t, a, c, b) to keep multi-join queries readable.*

UNION / UNION ALL

Combine results from multiple queries

```
SELECT 'Savings' as type, account_no, balance FROM savings_accounts
UNION ALL
SELECT 'Checking', account_no, balance FROM checking_accounts
ORDER BY balance DESC;
```

i *UNION removes duplicates (slower). UNION ALL keeps duplicates (faster).*

6

Index Commands

Index

CREATE INDEX

Speed up query performance

```
CREATE INDEX idx_account_status ON accounts(status);
CREATE INDEX idx_customer_name ON customers(last_name, first_name);
CREATE UNIQUE INDEX idx_account_no ON accounts(account_no);
CREATE INDEX idx_txn_date ON transactions(created_at);
CREATE INDEX idx_amount_date ON transactions(amount, created_at);
```

i Add indexes on columns used in *WHERE*, *JOIN ON*, and *ORDER BY*. Do not over-index -- each index slows *INSERT/UPDATE*.

SHOW / DROP INDEX

View and remove indexes

```
SHOW INDEX FROM accounts;
SHOW INDEX FROM transactions;
DROP INDEX idx_account_status ON accounts;
ALTER TABLE accounts DROP INDEX idx_old_index;
```

EXPLAIN

Analyze query execution plan

```
EXPLAIN SELECT * FROM accounts WHERE status = 'ACTIVE';

EXPLAIN SELECT a.*, c.name
  FROM accounts a JOIN customers c ON a.customer_id = c.id
 WHERE a.balance > 5000;

EXPLAIN FORMAT=JSON SELECT * FROM transactions
 WHERE created_at > DATE_SUB(NOW(), INTERVAL 30 DAY);
```

i Look at 'type': *ALL* = full table scan (bad), *ref/eq_ref* = index used (good). 'rows' shows estimated rows scanned.

CREATE USER

Create a new MySQL user

```
CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'StrongPass123!';  
CREATE USER 'appuser'@'%' IDENTIFIED BY 'StrongPass123!';  
CREATE USER 'readonly'@'localhost' IDENTIFIED BY 'ReadPass456!';
```

i '%' allows connection from any host. 'localhost' restricts to local only. Always create app-specific users -- never use root.

GRANT / REVOKE

Give or remove user permissions

```
GRANT SELECT, INSERT, UPDATE, DELETE ON bank_db.* TO 'appuser'@'localhost';  
GRANT SELECT ON bank_db.* TO 'readonly'@'localhost';  
GRANT ALL PRIVILEGES ON bank_db.* TO 'adminuser'@'localhost';  
FLUSH PRIVILEGES;  
  
REVOKE DELETE ON bank_db.* FROM 'appuser'@'localhost';  
SHOW GRANTS FOR 'appuser'@'localhost';
```

i In a banking app give the app user only SELECT/INSERT/UPDATE/DELETE -- never GRANT or DROP.

DROP USER / ALTER USER

Delete or modify users

```
DROP USER 'olduser'@'localhost';  
ALTER USER 'appuser'@'localhost' IDENTIFIED BY 'NewPass789!';  
ALTER USER 'appuser'@'localhost' PASSWORD EXPIRE;  
SELECT User, Host FROM mysql.user;
```

8

Transactions

Transaction

BEGIN / COMMIT / ROLLBACK

Control transaction boundaries

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE account_no = 'ACC001';
UPDATE accounts SET balance = balance + 500 WHERE account_no = 'ACC002';

-- Commit or rollback based on result
COMMIT; -- or ROLLBACK;
```

i Always wrap money transfers in a transaction. Either both updates succeed or both are rolled back.

SAVEPOINT

Create checkpoints within a transaction

```
START TRANSACTION;

INSERT INTO transactions (account_id, amount) VALUES (1, -500);
SAVEPOINT after_debit;

INSERT INTO transactions (account_id, amount) VALUES (2, 500);
SAVEPOINT after_credit;

ROLLBACK TO SAVEPOINT after_debit;

COMMIT;
```

i SAVEPOINTS let you roll back to a specific point without undoing the whole transaction.

SELECT FOR UPDATE / LOCK

Prevent concurrent modification

```
LOCK TABLES accounts WRITE;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UNLOCK TABLES;

-- Row-level locking (preferred)
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
SELECT * FROM accounts WHERE id = 1 LOCK IN SHARE MODE;
```

i FOR UPDATE locks selected rows until the transaction commits -- use for read-then-update patterns in banking.

SHOW STATUS / VARIABLES

View server status and config

```
SHOW STATUS;  
SHOW STATUS LIKE 'Threads%';  
SHOW VARIABLES LIKE 'max_connections';  
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';  
SHOW PROCESSLIST;  
SHOW FULL PROCESSLIST;
```

i *SHOW PROCESSLIST shows currently running queries -- useful to find slow or stuck queries in production.*

BACKUP / RESTORE

Export and import database data

```
-- Run in terminal (not MySQL shell)  
mysqldump -u root -p bank_db > bank_db_backup.sql  
mysqldump -u root -p bank_db accounts transactions > partial.sql  
  
-- Import / restore  
mysql -u root -p bank_db < bank_db_backup.sql  
  
-- Export as CSV  
SELECT * FROM accounts INTO OUTFILE '/tmp/accounts.csv'  
FIELDS TERMINATED BY ',' ENCLOSED BY ''''  
LINES TERMINATED BY '\n';
```

i *Schedule daily mysqldump backups. In banking environments test restores regularly.*

```
DELIMITER //
CREATE PROCEDURE TransferFunds(
  IN from_acc VARCHAR(20),
  IN to_acc   VARCHAR(20),
  IN amount   DECIMAL(15,2)
)
BEGIN
  DECLARE from_balance DECIMAL(15,2);
  SELECT balance INTO from_balance
    FROM accounts WHERE account_no = from_acc FOR UPDATE;
  IF from_balance >= amount THEN
    UPDATE accounts SET balance = balance - amount
      WHERE account_no = from_acc;
    UPDATE accounts SET balance = balance + amount
      WHERE account_no = to_acc;
    COMMIT;
  ELSE
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Insufficient funds';
  END IF;
END //
DELIMITER ;

CALL TransferFunds('ACC001', 'ACC002', 500.00);
```

i *Stored procedures run inside the DB -- faster than multiple round trips from your Spring Boot app.*

TRIGGER

Auto-run logic on table events

```
CREATE TRIGGER before_account_update
BEFORE UPDATE ON accounts
FOR EACH ROW
BEGIN
    IF NEW.balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Balance cannot be negative';
    END IF;
END;

CREATE TRIGGER after_transaction_insert
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    INSERT INTO audit_log(table_name, action, record_id, changed_at)
    VALUES ('transactions', 'INSERT', NEW.id, NOW());
END;
```

i Triggers are great for audit logging in banking -- record every change without relying on application code.

VIEW

Save a query as a virtual table

```
CREATE VIEW active_account_summary AS
SELECT
    c.name as customer_name,
    a.account_no, a.balance, a.status,
    COUNT(t.id) as transaction_count
FROM accounts a
JOIN customers c ON a.customer_id = c.id
LEFT JOIN transactions t ON a.id = t.account_id
WHERE a.status = 'ACTIVE'
GROUP BY a.id;

SELECT * FROM active_account_summary WHERE balance > 5000;
DROP VIEW active_account_summary;
```

i Views simplify complex queries for reporting. They do not store data -- they run the underlying query each time.

Category	Key Commands
Database	CREATE DATABASE, USE, SHOW DATABASES, DROP DATABASE
Table	CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE, DESCRIBE
CRUD	INSERT, SELECT, UPDATE, DELETE, ON DUPLICATE KEY UPDATE
Query	WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, CASE WHEN, Subqueries
Joins	INNER JOIN, LEFT JOIN, Multiple JOINS, UNION, UNION ALL
Index	CREATE INDEX, SHOW INDEX, DROP INDEX, EXPLAIN
User	CREATE USER, GRANT, REVOKE, FLUSH PRIVILEGES, DROP USER
Transaction	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT, FOR UPDATE
Admin	SHOW PROCESSLIST, mysqldump, Stored Procedures, Triggers, Views